

Software Test Plan (STP)

Project: Snake Game

Language: C/C++

Interface: SFML / Console-based

Version: 1.0

Team Number: 3

Team Details:

Name	SRN	Contribution
Jaanhavi Ashwin Kher	PES2UG24CS19 9	External Interface Requirements + RTM
KRISHNA PRAJOTH G	PES2UG24CS23 7	INTRO AND SYSTEM FEATURES
Krithika.S	PES2UG24CS23 8	Overall Description and UML use case diagram
Joey Fedrick K	PES2UG24CS20 6	NFR , Quality attributes & Acceptance tests

1. Introduction

Purpose: This document defines the test plan for the Snake Game v1.0. It outlines the objectives, scope, strategy, resources, schedule, and responsibilities for testing the system as specified in the Snake Game SRS.

Scope: Testing covers all functional features defined in SRS Section 4 (game initialization, snake movement and control, food management, collision detection and Game Over, difficulty/speed scaling, pause/resume, restart/exit, and score/Game State display), the non-functional requirements in Section 5 (performance, reliability, usability, portability, maintainability), and the security requirements in Section 5.1.2 (input validation and state-integrity checks). These features are organized by module as Test Items in Section 2, and restated by requirement ID in Section 3 for direct

alignment with the SRS and the RTM (SRS Section 8). Network connectivity, multiplayer functionality, database integration, and online leaderboard functionality are explicitly out of scope, consistent with the SRS's fixed project decisions.

References: Snake Game SRS v1.0 (Approved) — Sections 4, 5, and 8 (Requirements Traceability Matrix); the team's TC-[FUNCTION]-[NUMBER] test case naming convention (SRS Section 6.2, reused in Section 13 of this document).

Definitions: Player, Snake, Food, Game Board, Score, Game Over, Game State, Grid-based Movement, Collision, Pause, Restart — as defined in SRS Section 1.4. Additionally: SRS (Software Requirements Specification), RTM (Requirements Traceability Matrix), TC (Test Case).

2. Test Items

The following modules and features of the Snake Game system are identified as test items for this Test Plan. Each item corresponds to a conceptual module and/or functional area defined in the Snake Game SRS (Sections 2.2, 4, and 4.9), and will be verified against its related requirement IDs (SG-F-###, SGNF-###, SG-SR-###).

Test Item	Description	Related Req ID(s)
Game Controller	Game start, restart, and exit control flow.	SG-F-001, SG-F-014, SG-F-015
Input Handler	Keyboard input capture, direction control, immediate-reverse prevention, and input validation.	SG-F-002, SG-F-004, SG-SR-001, SG-SR-002
Snake Manager	Snake movement on the grid and growth on food collection.	SG-F-003, SG-F-007
Food Manager	Random valid food placement, ensuring food does not overlap the snake.	SG-F-005
Collision Manager	Detection of food collection, boundary collision, and self-collision.	SG-F-006, SG-F-009, SG-F-010
Score Manager	Score increment and score display.	SG-F-008, SG-F-016

Game State Manager	State transitions between Menu, Playing, Paused, and Game Over.	SG-F-011, SG-F-013, SG-F-017
Difficulty Manager	Speed scaling as the Player's score increases.	SG-F-012
Game Renderer / UI Screens	Main Menu, Gameplay Screen, and Game Over Screen presentation.	SG-F-016, SG-F-017
Security / Input Validation	Validation and rejection of unsupported or invalid input; state integrity on initialization.	SG-SR-001 to SG-SR-005

Non-functional characteristics of the above items — performance, reliability, usability, portability, and maintainability — are also in scope as test items, per SGNF-001 to SGNF-005 (Section 5 of the SRS).

3. Features to be Tested

The Test Items in Section 2 group testing scope by module; this section restates the same scope grouped by SRS requirement ID, so it maps directly onto the RTM (SRS Section 8) and the traceability tables in Section 13.

Functional (Section 4):

- SG-F-001 — Start new game (*Game Controller*)
- SG-F-002, SG-F-003, SG-F-004 — Directional input, continuous movement, immediate-reverse prevention (*Input Handler, Snake Manager*)
- SG-F-005, SG-F-006, SG-F-007, SG-F-008 — Food generation, collection detection, snake growth, score increase (*Food Manager, Collision Manager, Snake Manager, Score Manager*)
- SG-F-009, SG-F-010, SG-F-011 — Boundary collision, self-collision, transition to Game Over (*Collision Manager, Game State Manager*)
- SG-F-012 — Speed increase with score (*Difficulty Manager*)
- SG-F-013 — Pause and resume (*Game State Manager*)
- SG-F-014, SG-F-015 — Restart after Game Over, exit application (*Game Controller*)
- SG-F-016, SG-F-017 — Current score display, current Game State display (*Score Manager, Game Renderer / UI Screens*)

Non-Functional (Section 5):

- SGNF-001 — Input response ≤ 100 ms
- SGNF-002 — Consistent Game State, no crashes
- SGNF-003 — Clear, understandable controls/status
- SGNF-004 — Runs on supported desktop environment
- SGNF-005 — Modular, maintainable code

Security (Section 5.1.2):

- SG-SR-001, SG-SR-002 — Input validation, rejection of unrecognized commands
- SG-SR-003 — Prevention of unsupported input directly modifying Game State
- SG-SR-004 — No network transmission of gameplay data
- SG-SR-005 — Game variables initialized before gameplay begins

All 27 requirements above are covered end-to-end in the RTM (SRS Section 8), extended with test-execution status as described in Section 13 of this document.

4. Features Not to be Tested

Consistent with the SRS's fixed project decisions (Sections 1.2, 2.4, 2.5, 3.3, 3.4), the following are out of scope because they are not implemented in the system:

- Network connectivity / online communication — no data is transmitted over a network (Sections 2.4, 3.4).
- Multiplayer functionality — single Player only; no multiplayer mode (Sections 1.2, 2.5).
- Database integration / persistent storage — no database or external storage; Game State is held in memory only (Section 2.4).
- Online leaderboard functionality — not implemented (Section 1.2).
- Backend, server-side, or cloud interfaces — none exist (Section 3.3).
- Third-party API / external service integrations — not applicable (Section 3.3).
- Specialized or custom hardware peripherals — only a standard keyboard and display are supported (Section 3.2).
- Administrator/operator interfaces — single user role (Player) only; no admin/operator actor (Section 2.3).
- Compatibility with unsupported desktop environments — only the specified supported platform is in scope for portability testing (SGNF-004).

5. Test Approach / Strategy

Levels:

- Unit tests (module-level) — Game Controller, Input Handler, Snake Manager, Food Manager, Collision Manager, Score Manager, Game Renderer, Game State Manager, Difficulty Manager.
- Integration tests — e.g. Input Handler → Snake Manager → Collision Manager → Game State Manager; Food Manager → Snake Manager → Score Manager.
- System tests (end-to-end) — full gameplay cycle per Section 4.9.
- Acceptance tests (UAT-style) — all high-priority requirements (SG-F-001–017) and Section 6 quality attributes.

Types:

- Functional testing — SG-F-001 to SG-F-017.
- Regression testing — movement, collision, scoring, speed scaling after each fix.
- Performance testing — input response ≤ 100 ms (SGNF-001).
- Usability testing — clarity of controls/Game State (SGNF-003).
- Portability testing — runs on the supported desktop environment (SGNF-004).
- Maintainability review — modular, independently modifiable components (SGNF-005).

Entry Criteria:

- Stable build with all nine modules implemented.
- Approved SRS v1.0 available as baseline.
- Test cases for all SG-F, SGNF, SG-SR IDs designed and reviewed.
- Test environment (supported OS + SFML/console runtime) ready.

Exit Criteria:

- 100% of planned test cases executed.
- 0 critical/high-priority defects open.
- All SRS Section 6.1 acceptance criteria satisfied.
- RTM (SRS Section 8) shows Passed for all high-priority requirements.

5.1 Security Validation

- Validate Player input before movement processing (SG-SR-001).
 - Ignore invalid/unrecognized input without affecting Game State (SG-SR-002).
 - Prevent unsupported input from directly modifying Game State (SG-SR-003).
 - Confirm no gameplay data is transmitted over a network (SG-SR-004).
 - Verify game variables initialize to defined values before gameplay (SG-SR-005).
 - Boundary/invalid-input testing of keyboard handling for state-corruption checks.
-

6. Test Environment

The Snake Game shall be tested in the specified supported desktop environment using the C/C++ implementation. The graphical build shall use SFML, with a console-based interface as an alternative build target. The system is standalone and does not require network connectivity or a database.

Hardware

Environment Item	Specification / Purpose
Desktop / Laptop Computer	Used to build and execute the Snake Game.
Keyboard	The only supported input device for Player controls.
Display / Monitor	Used to display the game interface, score, and game-state information.

Software

Software	Purpose
C/C++	Implementation language of the Snake Game.
SFML	Used for graphical rendering of the game.
Console Interface	Alternative interface/build target supported by the project.
Supported Desktop Environment	Environment on which the application shall launch and perform core gameplay functions.

Test Conditions

Testing shall cover the defined game states and supported gameplay operations, including:

- Starting a new game.
- Snake movement using the supported Arrow keys.
- Prevention of immediate reverse direction.
- Food generation and collection.
- Snake growth and score increase.
- Boundary collision.
- Self-collision.
- Game Over.
- Speed increase as the score increases.
- Pause and resume using P.
- Restart using R after Game Over.
- Exit using Esc.
- Validation and rejection of invalid or unsupported input.
- Initialization of game variables before gameplay begins.

The Snake Game operates as a standalone, offline, single-player desktop application. No network communication, database, multiplayer mode, or online leaderboard is used.

7. Test Schedule

Testing will be carried out in stages as the Snake Game is developed and integrated. The schedule is organized according to the development and verification stages of the Snake Game.

Test Stage	Main Activity
Requirement and Test Case Preparation	Prepare test cases corresponding to the functional, non-functional, and security requirements defined in the SRS.
Module Testing	Verify the individual game components such as Game Controller, Input Handler, Snake Manager, Food Manager,

	Collision Manager, Score Manager, Game State Manager, Difficulty Manager, and Game Renderer.
Integration Testing	Verify the interaction between the major game components during gameplay.
System Testing	Execute the defined functional, non-functional, and security test cases on the complete Snake Game.
Acceptance Testing	Verify the acceptance criteria defined for the requirements and quality attributes.
Final Test Review	Review test results, requirement coverage, and remaining failures before final project submission.

Testing shall continue until the Exit Criteria defined in Section 5 are satisfied.

8. Test Deliverables

The following artefacts will be produced and maintained as part of the Snake Game testing effort:

- Test Plan — this document.
- Test Cases — manual test cases for each functional, non-functional, and security requirement, named per the team's TC-[FUNCTION]-[NUMBER] convention (e.g. TC-START-01, TC-MOVE-01, TC-COLL-01).
- Test Scripts — any automated or semi-automated test scripts used to exercise Snake Manager, Collision Manager, and Score Manager logic during unit/integration testing.
- Test Data — predefined game-state scenarios used for repeatable testing (e.g. fixed snake positions, fixed food-seed values, boundary edge cases, near-self-collision layouts).
- Test Execution Logs — records of each test run, including build version, date, and pass/fail outcome.
- Defect Reports — logged issues with steps to reproduce, severity, and the module/Req ID affected.
- Updated Requirements Traceability Matrix (RTM) — the RTM defined in SRS Section 8, extended with test execution Status (N / P / F) as testing proceeds.
- Test Summary Report — a final report summarising test coverage, pass/fail counts, and outstanding defects at the end of the test cycle.

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Joey Fedrick K (PES2UG24CS206)	Prepare and maintain the test plan; coordinate test execution; own non-functional and acceptance-test design, building on the NFR/Quality Attributes work in the SRS.
Test Engineer — Functional & System Testing	Krishna Prajoth G (PES2UG24CS237)	Design and execute functional test cases for System Features (SRS Section 4); log and track defects for SG-F-001–017.
Test Engineer — Traceability & Integration Testing	Jaanhavi Ashwin Kher (PES2UG24CS199)	Maintain the Requirements Traceability Matrix; design and execute interface/integration test cases, building on the External Interface Requirements + RTM work in the SRS.
Developer / Test Support	Krithika S (PES2UG24CS238)	Support defect fixes and triage; validate use-case-driven test scenarios against the UML use-case model.
Faculty / Evaluator	Prof. Sheela	Approve test results and sign off on test readiness, consistent with the Faculty/Evaluator stakeholder role defined in SRS Section 1.3.

10. Risks and Mitigation

Risk	Mitigation
SFML version/setup differs across team members' machines, causing build failures during testing	Standardize on one SFML version in the README's build instructions; provide the console-based fallback build as a backup test target

Grid-based movement edge cases (e.g., boundary wrap-around, off-by-one errors in grid coordinates) go unnoticed	Explicitly include boundary-value test cases in TC-COLL-01 and TC-MOVE-02, not just "happy path" movement
A defect in a core module (Snake Manager, Collision Manager, or Game State Manager) blocks more than 30% of planned test cases	Invoke the Suspension Criteria in Section 12 immediately rather than continuing partial testing; resume only once a smoke test confirms core gameplay works
Difficulty scaling (SG-F-012) makes gameplay untestable at high scores due to speed exceeding input response time	Cap maximum speed during testing or test SGNF-001 (input response) independently of live difficulty scaling
Parallel development across four contributors on separate modules (per Section 2's Test Items) introduces integration bugs at merge time	Require test cases relevant to a module to pass locally before a PR is opened; re-run integration tests after each merge into main
Limited testing time before submission deadline	Prioritize High-priority functional requirements (per SRS Section 4) first; Medium-priority items (e.g., SG-F-012, SG-F-013) tested afterward if time allows

11. Assumptions & Dependencies

- Build under test implements all 17 functional requirements (SG-F-001–017).
- Approved SRS v1.0 is the single source of truth; changes require updating the master decision sheet first (SRS Section 2.5).
- Supported desktop environment (SRS Section 2.4) is available for portability testing (SGNF-004).
- SFML library / console runtime is correctly installed in the test environment.
- No network, database, or external service required (SRS Sections 2.4, 3.3, 3.4).
- Standard keyboard and display available; no specialized hardware needed (SRS Section 3.2).
- Test cases follow the TC-[FUNCTION]-[NUMBER] convention (SRS Section 6.2).
- RTM (SRS Section 8) stays up to date for requirement-to-test-case-to-status mapping.

- Team members available throughout the test cycle per roles in Section 9.
-

12. Suspension & Resumption Criteria

Suspend testing if:

- A build fails to launch, or crashes on startup, blocking execution of the planned test cases.
- A defect in a core module (Snake Manager, Collision Manager, or Game State Manager) blocks more than 30% of the planned test cases from being executed.
- The test/build environment (compiler toolchain, SFML setup, or console-based build target) becomes unavailable or unusable.
- A critical defect causes repeated, unrecoverable crashes during normal gameplay (violating SGNF-002, Reliability).

Resume testing if:

- The blocking defect has been fixed by the development team and verified in a new build.
 - The test/build environment has been restored and is stable.
 - A smoke test confirms that core gameplay (start, move, collide, Game Over) functions correctly before full test execution resumes.
-

13. Test Case Management & Traceability

Test case management for the Snake Game follows the team's agreed naming convention: TC-[FUNCTION]-[NUMBER] (e.g. TC-START-01, TC-MOVE-01, TC-FOOD-01, TC-COLL-01, TC-SCORE-01, TC-PAUSE-01, TC-RESTART-01, TC-PERF-01, TC-SEC-01). Every functional (SG-F-###), non-functional (SGNF-###), and security (SG-SR-###) requirement is traced to its owning SRS section, its responsible module, and its corresponding test case in the Requirements Traceability Matrix (RTM), first defined in SRS Section 8 and reused/extended here for test execution tracking.

Representative traceability examples:

Req ID	Requirement (short)	Module	Test Case
--------	---------------------	--------	-----------

SG-F-001	Start new game	Game Controller	TC-START-01
SG-F-005	Generate food at valid position	Food Manager	TC-FOOD-01
SG-F-009	Detect boundary collision	Collision Manager	TC-COLL-01
SG-F-011	Transition to Game Over	Game State Manager	TC-GAMEOVER-01
SGNF-001	Input response ≤ 100 ms	Input Handler / Game Renderer	TC-PERF-01
SG-SR-001	Validate Player input before movement	Input Handler	TC-SEC-01

The full RTM (all 17 functional, 5 non-functional, and 5 security requirements) is maintained in the SRS (Section 8) and will be updated with a Status column (N = Not tested, P = Passed, F = Failed) as test execution proceeds during this test cycle.

14. Test Metrics & Reporting

Testing progress and results shall be measured using the requirements, acceptance criteria, quality attributes, and test cases defined in the SRS.

Metric	Measurement / Criteria
Requirement Verification	Percentage of defined requirements verified through their corresponding test cases.
Test Execution	Number and status of defined test cases executed.
Functional Test Results	Pass/fail results for the defined SG-F-### functional requirements.
Performance	Valid Player input response shall be ≤ 100 ms under normal conditions.

Reliability	The game shall maintain a consistent Game State and complete defined gameplay tests without an unexpected crash.
Usability	Player shall be able to identify controls and game status without additional technical instructions.
Portability	The application shall launch and perform core gameplay functions on the specified supported desktop environment.
Maintainability	Major game components shall be modular and logically organized.
Security Verification	All five security requirements, SG-SR-001 to SG-SR-005, shall be verified using their corresponding security tests.
Acceptance Criteria	All defined acceptance tests shall be verified against their corresponding requirements.

The SRS defines the corresponding quality-attribute test cases as TC-PERF-01, TC-REL-01, TC-UX-01, TC-PORT-01, TC-MAINT-01, and TC-SEC-01 to TC-SEC-05.

Test Reports

The following testing records shall be maintained:

- Test Case Results - records the result of each defined test case.
- Requirements Traceability Matrix (RTM) - links requirements to their corresponding test cases.
- Acceptance Test Results - records verification of the defined acceptance criteria.

The final testing review shall confirm that the Exit Criteria defined in Section 5 have been met.